SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
ASSESSMENT TOOL 1 – Analytical Problem Solving

Course Code:	CSA03	**Course Title:**	Data Structures for Social Networks
CO Assessed:	CO2 – Imitation (BL3, BL4)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	15% of CIA	**Total Marks:**	50
CO1 Statement:	ADT array, Operations on arrays, Searching Algorithms, Linear Search, Binary Search, Suffix Arrays & LCP Arrays ADT, Linked List, Stack, Queue, Applications.		
Student Name:	**T.VAISHNAVI**	**Reg. No.:**	**192421370**
Section / Batch:	**B-TECH(IT)**	**Date:**	**27/07/2026**

Code of Conduct

I **T.VAISHNAVI(192421370)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Analytical Problem solving questions, Total: 50 Marks.
- Evaluation of Answer Quality -Checking whether the answer includes: Problem Understanding, Method Selection, Solution Process, Accuracy of Calculation, Interpretation of Results
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
ADT array, Operations on arrays, Searching Algorithms, Linear Search, Binary Search,	Array ADT, Linear & Binary Search Algorithm	1, 2	20
Suffix Arrays & LCP Arrays ADT, Linked List	Linked List	3,4	20
Stack, Queue, Applications.	Application of Stack, Queue	5	10
	GRAND TOTAL:	**50Marks**	

Annexure A – Analytical Problem Solving

Q1) Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail. Design a system that dynamically chooses between linear search and binary search based on:

- Data size
- Sortedness
- Frequency of queries

Analyze decision criteria mathematically

Q2) Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail.

Q3) Pictorially insert an element at 3rd position in a doubly linked list, having 6 elements in the list and delete the element at 2nd position from the list. Implement and analyse it. Construct an efficient system using two stacks within a single array structure to maximize memory utilization. Compare fixed partitioning and dynamic allocation strategies, and analyze how each approach affects overflow conditions, space efficiency, and operational complexity under varying workloads.

Q4) Convert the following infix expression into a post fix expression. B - (C/D + (E % F* G) / H) * J. Also, evaluate the given postfix expressions. **a)** 9 3 8 4 /*+ b) 6 2 + 6 3 /* b) Convert the expression ((a + b) + c * (d + e) + f) * (g + h) to 1) Prefix expression , 2) Postfix expression.

Q5) Perform the following operations in Circular Queue having a size of 5. Enqueue(20), Enqueue(10), Enqueue(30), Dequeue(), Enqueue(80), Dequeue(), Enqueue(90), Enqueue(100), Dequeue(), Dequeue(), Enqueue(60), Enqueue(90). Finally display the front and rear elements with its positions. Discuss in detail.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

		relevance			

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

REDMI 13C 5G

CO2 - AT1

Analytical Problem Solving

Name:- T. Vaishnavi
Regno :- 192421370
Course code :- CSA0312
Course Name :- Data structure

① Given array

66, 111, 212, 323, 432, 543, 643, 649, 777, 838

First we need to find mid value

W.K.T

$$Mid = \frac{low + high}{2}$$

first lowest value = 66 ⇒ Index 0

highest value = 838 → Index 9

(i) case : key = 111

L → Low
H → High

L = 0
H = 9

$Mid = \frac{0+9}{2} = 9/2 = 4.5 = 4$

0	1	2	3	4	5	6	7	8	9
66	111	212	323	432	543	643	649	777	838

111 < 432

now, we need to change the high value

high = mid - 1 = 4 - 1 = 3

L = 0
H = 3

$Mid = 0+3/2 = 3/2 = 1.5 = 1$

Element = 111

66, 111, 212, 323, 432, 543, 643, 649, 777, 838

111 = 111

30/07/2026 18:09

The given element is found at index 1

case 2: key = 888

L = 0
H = 9

$Mid = \frac{Low + high}{2} = \frac{0+9}{2} = 4.5 = 4$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

wow the given value is greater than middle element so,

low = mid + 1
low = 5
high = 9

$mid = \frac{5+9}{2} = \frac{14}{2} = 7$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

888 > 649

So,

Low = mid + 1 = 7 + 1 = 8

L = 8
H = 9

$Mid = \frac{L+H}{2} = \frac{8+9}{2} = \frac{17}{2} = 8$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

888 > 777

L = mid + 1 = 8 + 1 = 9

L = 9
H = 9

$mid = \frac{9+9}{2} = \frac{18}{2} = 9$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

888 = 888

The given element is found at index 9

Case 3 :- key = 999 :

L = 0

H = 9

$M = \frac{0+9}{2} = 4.5 = 4$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

999 > 432

So,

L = mid + 1 = 4 + 1 = 5

$mid = \frac{5+9}{2} = \frac{14}{2} = 7$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

999 > 649

Low = mid + 1 = 7 + 1 = 8

L = 8

H = 9 $\quad mid = \frac{8+9}{2} = \frac{17}{2} = 8$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

999 > 777

L = mid + 1 = 8 + 1 = 9

$mid = \frac{9+9}{2} = \frac{18}{2} = 9$

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

999 > 888

Finally

The given element is not found

Linear search → upto 10 comparisons

Binary search → about 4 comparisons

Hence, for large data, Binary search is better.

(2) Given array:-

0	1	2	3	4	5	6	7	8	9
66	111	212	328	432	563	643	669	771	888

i) key = 111

Low	high	mid	mid element	Result
0	9	4	432	111 < 432 → high = 3
0	3	1	111	found in 2nd iteration

Index = 1

ii) key = 888

Low	high	mid	mid element	Result
0	9	4	432	888 > 7432
5	9	7	649	888 > 649
8	9	8	777	888 > 777
9	9	9	888	found

found at Index 9

iii) key = 999

Low	high	mid	mid element	result
0	9	4	432	999 > 432
5	9	7	649	999 > 649
8	9	8	777	999 > 777
9	9	9	888	999 > 888

low(10) > High(9)

∴ Element is not found

Analysis of Binary Search :-

→ It works on sorted arrays

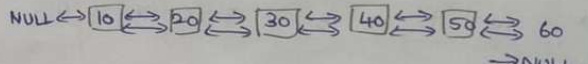
Time complexity

- Best case = O(1)
- average case = O(log 2n)
- worst case = O(log 2n)
- Space complexity = O(1)

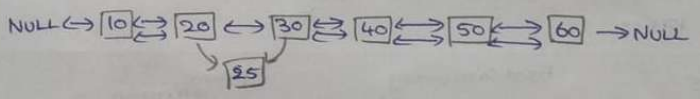
3) Doubly linked list

a) Insert an element at 3rd Position :-

Initial list (6 element)

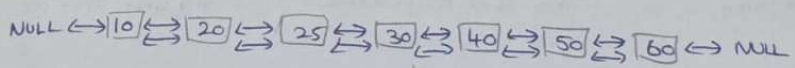


Insert 3rd position (element 25)



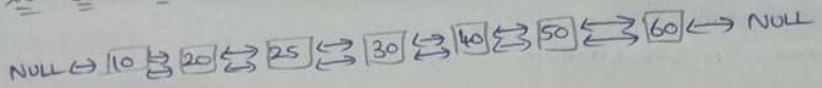
1. New = 25
2. New → prev = 20
3. new → next = 30
4. 20 → next = new
5. 30 → Prev = new

After insertion

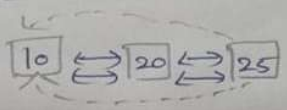


b) Delete the element at 2nd Position :

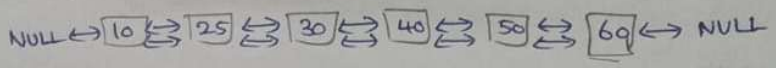
list before deletion :-



Delete 20 (2nd Position)



After deletion



Two stacks in a single array

Array size = 10

Index	0	1	2	3	4	5	6	7	8	9
Initial	-	-	-	-	-	-	-	-	-	-

Stack 1 grows from left →

Stack2 grows from right

Comparison :	Fixed Partitioning	Dynamic allocation
memory division	Array divided equally into two parts	Both
Space utilization	Lower	higher
flexibility	one stack may overflow even other has space	only when Top1 + Top2
Push/Pop complexity	O(1)	O(1)

④ B-(C/D+(E%F*G/H)*I)

Symbol	Postfix string	Stack
B	B	-
-	B	-C
(	B	-C/
C	BC	-C/
/	BC	-C/+
D	BCD	-C+/
+	BCD/	-C+(
E	BCD/E	-(+(%
%	BCD/E	-(+(%
F	BCD/EF	-(+(*
*	BCD/EF%	-(+(*
G	BCD/EF%G	-(+
)	BCD/EF%G*	-C+/
/	BCD/EF%G*H	-C+/
H	BCD/EF%G*H	-
)	BCD/EF%G*H/+	-*
*	BCD/EF%G*H/+	-*
I	BCD/EF%G*H/+I	-*
End	BCD/EF%G*H/+I-	

a) 9384/*+

Symbol	Stack operation	Stack
9	Push 9	9
3	Push 3	9,3
8	Push 8	9,3,8
4	Push 4	9,3,8,4
/	8/4=2	9,3,2
*	3x2=6	9,6
+	9+6=15	15

ans=15

b) Postfix expression

62+63/*

Symbol	Stack operation	Stack
6	Push 6	6
2	Push 2	6,2
+	6+2=8	8
6	Push 6	8,6
3	Push 3	8,6,3
/	6÷3=2	8,2
*	8x2=16	16

ans=16

⑤ circular queue (size = 5)

Queue size = 5

Position = 0, 1, 2, 3, 4

Operations

Enqueue (20), Enqueue (10), Enqueue (30), Dequeue (), Enqueue (80), Dequeue ()

Enqueue (90), Enqueue (100), Dequeue (), Dequeue (), Enqueue (60), Enqueue (90)

Step	Operation	front	rear	Queue Status
Initial	Queue Empty	-1	-1	[-, -, -, -, -]
1	Enqueue(20)	0	0	[20, -, -, -, -]
2	Enqueue(10)	0	1	[20, 10, -, -, -]
3	Enqueue(30)	0	2	[20, 10, 30, -, -]
4	Dequeue() remove 20	1	2	[-, 10, 30, -, -]
5	Enqueue(80)	1	3	[-10, 30, 80, -]
6	Dequeue()	2	3	(-, -, 30, 80, -]
7	Enqueue(90)	2	3	[-, -, 30, 80, -]
8	Enqueue(100)	2	0	[100, -, 30, 80, 90]
9	Dequeue ()	3	0	[100, -, -, 80, 90]
10	Dequeue()	4	1	[100, 60, -, -, 90]
11	Enqueue (60)	4	1	[100, 60, -, -, 90]
12	Enqueue (90)	4	2	[100, 60, 90, -, 90]

Position	0	1	2	3	4
Element	100	80	90	Emp	90

Front element = 4 = 90 (Position 4)

Rear = 2 = 90 (Position 2)